Edwin Antonie – LB01 – 2802397306

Bagian A

1. **Efek Regularisasi pada Batch Normalization (Slide 6–8)**

   Jawaban:

   Batch Normalization bekerja dengan menormalisasi aktivasi neuron ke sekitar nol menggunakan rata-rata dan varians per *mini-batch* (Sesi 5; Slide 6 – 7). Karena komposisi *sample* per *batch* acak, nilai normalisasi suatu data bisa bergeser bergantung kepada pasangan *sample*-nya. Fluktuasi pada nilai normalisasi ini membuat *noise* halus yang mencegah neuron untuk saling bergantung dan menghafal angka melainkan memunculkan efek regularisasi mirip *Dropout* (Slide 8).

   Namun, asumsi ”menghilangkan *Dropout*” ini invalid pada beberapa kondisi:

   1. **Ukuran *batch* terlalu kecil ($m \leq 4$):** membuat perhitungan statistik terlalu liar sehingga menyebabkan arah gradien kacau dan memicu instabilitas saat pelatihan.
   2. **Ukuran *batch* terlalu besar ($m \geq 512$):** membuat *noise* yang dihasilkan mendekati 0 sehingga kehilangan kemampuan untuk regularisasi dan membutuhkan *dropout* untuk mengembalikan efek regularisasi.
   3. **Arsitektur berbasis sekuensial (RNN, LSTM):** saat statistik data berubah-ubah, *Batch Normaliszation* tidak dapat diimplementasikan secara optimal pada kasus ini sehingga *dropout* tetap essensial untuk mengurangi *overfitting* pada model.

   Referensi PPT: Sesi 5, Slide 6 – 8

   Referensi Eksternal:

   https://medium.com/deep-learning-journal/day-11-all-about-dropout-regularization-in-neural-networks-b975c08bc07e

   https://medium.com/analytics-vidhya/everything-you-need-to-know-about-regularizer-eb477b0c82ba

   https://skirene.medium.com/demystifying-batch-normalization-vs-drop-out-1c8310d9b516

2. Degradasi Jaringan Dalam dan Formulasi Residual (Slide 10–12)

Jawaban:

Penambahan kedalaman lapisan ke *plain CNN* memicu *degradation problem* yakni model kesulitan belajar dikarenakan tumpukan perkalian matriks dan aktivasi non-linear yang kompleks membuat *optimizer* kesulitan dan gagal untuk menemukan fungsi identitas tersebut, sehingga menurunkan akurasi pada *training* (Sesi 5; Slide 10).

ResNet memacahkan masalah ini dengan teknik *skip connection* yakni dengan rumus $y = F(x) + x$ (Sesi 5; Slide 11). Dimana, lapisan yang tidak penting cukup mempelajari fungsi sisa yang bernilai 0 ($F(x) = 0$) sehingga dapat diteruskan tanpa perubahan nilai. Saat *back propagation*, turunan yang dihasilkan menjadi $\frac{\partial \mathcal{E}}{\partial x} = \frac{\partial \mathcal{E}}{\partial y}\left(\frac{\partial F(x)}{\partial x} + 1\right)$ dimana, gradien tidak harus melewati setiap lapisan secara berurutan, gradien dapat berjalan mundur secara langsung berkat komponen + 1. Hal ini membuat nilai gradien tetap cukup besar untuk memperbarui bobot lapisan awal mengatasi *vanishing gradient*.

Referensi PPT: Sesi 5, Slide 10 – 12

Referensi Eksternal:

https://www.datacamp.com/id/tutorial/resnet-architecture

3. Feature Extraction vs. Fine-Tuning (Slide 14–16)

Jawaban:

Feature Extraction membekukan seluruh *layer* parameter dari *model pre-trained* dan hanya melatih *classifier layer head* baru, sedangkan *Fine-Tuning* membuka sebagian *layer* di *model pre-trained* dan melatih model tersebut dengan *layer head* baru menggunakan *learning rate* yang rendah, perubahan ini membuat *weight* pada *model* sesuai dengan kebutuhan.

Dimensi Evaluasi	Feature Extraction	Fine-Tuning
Trainable Parameters	Sangat sedikit, karena hanya lapisan klasifikasi paling akhir yang dilatih.	Banyak, karena *convolution layer* dan *classifier* dilatih bersamaan.
Kebutuhan Data	Rendah, cocok untuk data terbatas karena minim risiko menghafal data.	Tinggi, rentan terkena *overfitting* apabila data *training* sedikit karena parameter-nya yang banyak.

Beban Komputasi	Ringan, karena perhitungan gradien hanya dilakukan pada lapisan akhir.	Berat, komputasi gradien harus dilakukan pada setiap *layer* saat melakukan *backpropagation*.
Risiko Fitur Rusak	Nol, representasi visual bawaan model tidak akan mengalami modifikasi.	Tinggi, karena saat model belajar informasi baru, perhitungannya dapat menggantikan fitur dari *model* awalnya (catastrophic forgetting)

Kegunaan masing-masing adalah: *feature extraction* jika data target sangat terbatas dan objeknya masih serupa dengan foto umum sehari-hari. Sedangkan, *fine-tuning* jika data cukup banyak serta membutuhkan model mengenali karakterisitik bentuk khusus yang lebih detail.

Referensi PPT: Sesi 5, Slide 14 – 16

Referensi Eksternal:

https://medium.com/@ayasc/transfer-learning-feature-extraction-vs-fine-tuning-3735eff6091e

4. Kaidah Penerapan Data Augmentation (Slide 20, 21, 23)

Jawaban:

Data Augmentation hanya diterapkan pada *train set* untuk memperbanyak variasi sampel belajar dan mencegah model untuk menghafal data (*overfitting*), sehingga kemampuan generalisasi model dapat meningkat (Sesi 5; Slide 20 – 21). Sedangkan, *validation set* dan *test set* harus terlepas dari manipulasi acak agar evaluasi performa model bisa objektif, konsisten, dan mencerminkan kondisi data asli di dunia.

Pada klasifikasi *pet breeds* augmentasi yang terlalu ekstrim seperti manipulasi warna(*grayscale*, *color jittering*, *hue inversion*) tidak layak diterapkan dikarenakan corak dan warna bulu merupakan fitur pembeda utama antar ras-hewan tersebbut. Menghilangkan informasi warna ini dapat merusak karakteristik penting sehingga menyulitkan model dalam mengenali ras.

Referensi PPT: Sesi 5, Slide 20 – 21

Referensi Eksternal:

https://medium.com/@karthikmarimuthu2010/automating-data-augmentation-a-beginner-friendly-guide-for-deep-learning-6e58d1d4a926

5. **Mitigasi Catastrophic Forgetting (Slide 28–29)**

Jawaban:

Catastrophic Forgetting adalah kecenderungan jaringan neuron untuk melupakan informasi lama saat dilatih dengan data baru. Fenomena ini dipicu oleh *weight overwriting*, yakni pembaruan gradien pada data baru yang menggeser parameter terdahulu. Fenomena ini juga merusak representasi visual bawaan *pre-trained model* karena tidak-adanya mekanisme latihan ulang terhadap data masa lalu.

Dua strategi mitigasi *catastrophic forgetting*:

1. **Differential Learning Rate:** secara gradual meningkatkan *learning rate* yang awalnya rendah pada lapisan awal, perlahan semakin meningkat pada lapisan *classifier* baru. Langkah ini membatasi pergeseran *weight* pada fitur penting agar tidak tertimpa secara agresif.
2. **Gradual Unfreezing:** Membuka *freezing layer* secara bertahap dari atas ke bawah setelah lapisan klasifikasi stabil, memastikan gradien yang merambat ke *backbone* teratur dan tidak destruktif.

Referensi Eksternal:

https://medium.com/vannguardai/catastrophic-forgetting-and-continual-learning-10ec6c9bd7df
https://medium.com/slavv/differential-learning-rates-59eff5209a4f
https://www.emergentmind.com/topics/progressive-layer-unfreezing